Guía de Estudio Completa: Redes Neuronales Convolucionales (CNN)

Curso de Inteligencia Artificial — ESPOL FIEC · Prof. Enrique Peláez, Ph.D.

Preparación formal para examen (PAO 2026-I)

> Esta guía integra: (1) las anotaciones normalizadas de las clases 10, 11 y 12 (donde el profesor introduce, desarrolla y revisa las CNN); (2) la guía maestra del curso; (3) el banco de 150+ preguntas de exámenes anteriores («Preguntas de IA»); y (4) complementos formales tomados de la literatura estándar de aprendizaje profundo (Goodfellow, Bengio & Courville — *Deep Learning*; Brownlee; Hagan — *Neural Network Design*) para completar el rigor matemático donde las notas de clase son esquemáticas. Cada fórmula viene acompañada de al menos un ejercicio resuelto **idéntico en estructura** a los que ha usado el profesor en exámenes pasados.

Tabla de contenidos

1. Motivación y definición formal
2. La operación de convolución
3. Hiperparámetros de una capa convolucional: stride y padding
4. Fórmula de tamaño de salida (volumen)
5. Número de parámetros de una capa convolucional
6. Capas de pooling (submuestreo)
7. Convoluciones 1×1
8. Por qué funciona una CNN: conexiones dispersas, pesos compartidos y equivarianza traslacional
9. Arquitectura completa de una CNN
10. Forward propagation formal
11. Backpropagation en una CNN (incluye la máscara de pooling)
12. Comparación cuantitativa MLP vs. CNN
13. Banco de ejercicios resueltos (todos los patrones del examen)
14. Preguntas conceptuales verdadero/falso explicadas
15. Hoja de fórmulas (cheat-sheet) para el examen
16. Errores comunes y trampas del profesor
17. Glosario formal obligatorio

---

1. Motivación y definición formal

1.1 Definición

Una **red neuronal convolucional** (*Convolutional Neural Network*, CNN) es una arquitectura de red neuronal diseñada para datos con estructura espacial (o topológica) — típicamente imágenes — que sustituye la conexión densa («todos contra todos») de la MLP por una operación de **convolución**: un filtro (o *kernel*) de pesos pequeño se desliza sobre la entrada realizando **productos punto locales**.

Formalmente, mientras que en una capa densa de una MLP cada neurona de salida es una combinación lineal de **todas** las entradas:

```
z_j = Σ_i  w_ij · y_i          (conexión densa: "producto cruz")
```

en una capa convolucional cada valor de salida es una combinación lineal de solo una **ventana local** de la entrada, usando **los mismos pesos** en cada posición:

```
z(i,j) = Σ_u Σ_v Σ_c  K(u,v,c) · X(i+u, j+v, c) + b          (convolución:
"producto punto local")
```

1.2 Motivación biológica (Hubel–Wiesel y el Neocognitron)

- **Hubel y Wiesel** (corteza visual de gatos): descubrieron neuronas que se activan únicamente ante estímulos visuales en una **región acotada del campo visual** (campo receptivo), y que distinguen entre:
  - **Células simples**: responden a patrones orientados específicos (bordes, líneas) en una posición fija.
  - **Células complejas**: responden al mismo patrón **independientemente de su posición exacta** dentro de una región (invarianza local a desplazamiento).
- **Fukushima — Neocognitron**: primer modelo computacional inspirado en lo anterior, con arquitectura tipo «sándwich»:
  - **Capas S** (*simple*): equivalentes a los filtros convolucionales actuales (parámetros modificables/aprendibles).
  - **Capas C** (*complex*): equivalentes al pooling actual (submuestreo, sin parámetros aprendibles).

> **Correspondencia biológica → CNN (tema de examen):**
> - Células simples → filtros/kernels detectores de patrones.
> - Células complejas → capas de pooling (invarianza parcial a desplazamiento).
> - Campo receptivo → ventana espacial cubierta por el filtro.

1.3 Tareas típicas de visión que motivan la CNN

Clasificación y recuperación de imágenes, detección y segmentación de objetos, estimación de posturas/movimiento/intención/ruta, generación de descripciones (*captioning*) y etiquetado (*tagging*). Una MLP no puede relacionar de forma natural varios *frames* de una secuencia visual; una CNN, en cambio, explota la correlación espacial (y con extensiones temporales, secuencial) de los datos.

---

2. La operación de convolución

2.1 Definición matemática general

Dada una entrada (volumen) `X` de dimensiones `n_H × n_W × n_C` (alto × ancho × profundidad/canales) y un filtro `K` de dimensiones `F × F × n_C` (el filtro **siempre** tiene la misma profundidad que la entrada), la activación de salida en la posición `(i,j)` es:

```
z(i,j) = Σ_{u=1}^{F} Σ_{v=1}^{F} Σ_{c=1}^{n_C}  K(u,v,c) · X(i+u, j+v, c)  +
b
```

donde:

Símbolo	Significado
`X`	Entrada (imagen o mapa de activación de la capa anterior)
`K`	Filtro / kernel (contiene los pesos `W` que se aprenden)
`F`	Tamaño espacial del filtro (se asume cuadrado en este curso: `F × F`)
`n_C`	Profundidad de la entrada (número de canales)
`b`	Sesgo (*bias*), uno por filtro
`z(i,j)`	Activación resultante en la posición `(i,j)` del mapa de salida

Nota terminológica importante (marcada por el profesor): lo que en visión por computador se llama estrictamente «convolución» matemática incluye una inversión (*flip*) del kernel; lo que realmente implementan las CNN modernas (y lo que se enseñó en clase) es una **correlación cruzada**. En la práctica de este curso, y en toda la literatura de *deep learning*, se sigue llamando «convolución» a esta operación sin inversión — es la convención que debes usar en el examen.

2.2 Interpretación como perceptrón local

Cada aplicación del filtro en una posición es matemáticamente idéntica a un **perceptrón**: un

producto punto entre un vector de pesos (el filtro «aplanado») y un vector de entradas (la ventana local «aplanada»), más un sesgo, seguido de una función de activación:

```
a(i,j) = f( z(i,j) ) = f( K · X_ventana + b )
```

La diferencia esencial con la MLP es que **ese mismo perceptrón (mismos pesos `K`, mismo bias `b`) se reutiliza en todas las posiciones de la imagen** — de aquí nace el concepto de **pesos compartidos** (sección 8).

2.3 Multiplicidad de filtros

Una capa convolucional no usa un único filtro, sino `K_j` filtros (hiperparámetro de la capa `j`). Cada filtro:

- Tiene la **misma profundidad** que el volumen de entrada.
- Se **especializa** en detectar un patrón distinto (bordes verticales, horizontales, texturas, colores, y en capas profundas, patrones semánticos cada vez más abstractos).
- Produce **un mapa de activación (feature map)** propio.

Los `K_j` mapas de activación se **apilan** en la dimensión de profundidad, formando el volumen de salida de la capa. Por eso:

```
profundidad de salida de la capa j  =  número de filtros de la capa j  =  K_j
```

2.4 Ejemplo elemental (ilustrativo, filtro 3×3, entrada 5×5×1, sin padding, stride 1)

```
Entrada X (5×5):            Filtro K (3×3):        b = 0
1 2 3 0 1                   1  0 -1
0 1 2 3 0                   1  0 -1
1 0 1 2 1                   1  0 -1
2 1 0 1 2
0 2 1 0 1
```

Cálculo de `z(1,1)` (ventana superior-izquierda, posiciones 1..3 en ambos ejes):

```
Ventana:
1 2 3
0 1 2
1 0 1

z(1,1) = (1·1)+(2·0)+(3·-1) + (0·1)+(1·0)+(2·-1) + (1·1)+(0·0)+(1·-1)
       = (1+0-3) + (0+0-2) + (1+0-1)
       = -2 + -2 + 0 = -4
```

Este procedimiento se repite deslizando el filtro por toda la imagen (según el stride), produciendo el mapa de activación completo. El tamaño de ese mapa se calcula formalmente en la sección 4.

---

3. Hiperparámetros de una capa convolucional: stride y padding

3.1 Stride (`S`)

Es el **tamaño del paso** con el que el filtro se desliza sobre la entrada (horizontal y verticalmente; en este curso se asume el mismo stride en ambas direcciones).

- **A mayor stride → menor mapa de activación de salida** (el filtro visita menos posiciones).
- Un stride `S = 1` produce el mapa de activación más grande posible (máximo solapamiento entre ventanas consecutivas).
- El profesor recalca que el stride debe ser **pequeño** para asegurar traslape suficiente entre ventanas consecutivas y no perder información espacial.

3.2 Padding (`P`)

Es el **relleno con ceros** (un «marco» de píxeles con valor 0) que se añade alrededor de la entrada antes de aplicar la convolución.

Dos motivaciones formales:

1. **Evitar que el filtro «se salga»** de los bordes de la imagen en ciertas combinaciones de `F` y `S`.
2. **Conservar el tamaño espacial** de la entrada tras la convolución («padding same», en Keras: `padding="same"`), evitando que la información de los bordes se use menos veces que la del centro y evitando que el volumen se reduzca capa tras capa.

Tipos de padding (terminología estándar, complementaria a la de clase):

Tipo	Descripción	Efecto en el tamaño
Valid (`P = 0`)	Sin relleno	El volumen se reduce en cada capa
Same	`P` calculado para que la salida tenga el mismo tamaño que la entrada (cuando `S = 1`)	El volumen se mantiene
Full	Relleno máximo, el filtro empieza a tocar solo una esquina	El volumen aumenta

> **Recomendación explícita del profesor:** «no calcular el padding manualmente en la práctica; dejar que el algoritmo lo determine automáticamente (`padding="same"`) para no perder información». Sin embargo, **en el examen sí debes saber calcularlo a mano** con la fórmula de la sección 3.3, porque es uno de los temas declarados «va a ser pregunta de examen».

3.3 Padding necesario para conservar el tamaño (`S = 1`)

Partiendo de la fórmula general de tamaño de salida (sección 4) e imponiendo `O = N`:

```
N = (N + 2P - F)/S + 1
```

Para `S = 1`, despejando `P`:

```
P = (F - 1) / 2
```

Regla práctica: para conservar el tamaño con stride 1, el padding es la mitad de `(F-1)`. Esta fórmula solo produce un entero cuando `F` es **impar** — por eso casi todos los filtros de CNN son de tamaño impar (3×3, 5×5, 7×7).

3.4 Padding para un tamaño de salida objetivo arbitrario (`O` dado, `S` cualquiera)

Despejando `P` de la fórmula general para un `O` objetivo cualquiera:

```
P = ( (O - 1)·S + F - N ) / 2
```

---

4. Fórmula de tamaño de salida (volumen)

4.1 Fórmula general (la fórmula más preguntada del curso)

Para una entrada espacial cuadrada `N × N` (o, de forma más general, `n_H × n_W`), un filtro `F × F`, padding `P` y stride `S`:

$$O = \frac{N + 2P - F}{S} + 1$$

Si la entrada no es cuadrada, la fórmula se aplica **por separado** a alto y ancho:

```
n_H_out = ⌊(n_H_in + 2P - F)/S⌋ + 1
n_W_out = ⌊(n_W_in + 2P - F)/S⌋ + 1
n_C_out = K        (K = número de filtros de la capa)
```

El volumen de salida completo es, por lo tanto:

```
n_H_out × n_W_out × K
```

> **Regla de consistencia (examen):** si el resultado de `(N + 2P - F)/S` no es un número entero, esa combinación de hiperparámetros **no es válida** — el filtro no encaja exactamente en la entrada con ese stride y padding.

4.2 Ejemplo resuelto (formato exacto de las notas de clase)

Datos: entrada `32×32×3`; capa con `K = 10` filtros de `F = 5×5`; `S = 1`; `P = 2`.

```
O = (32 + 2·2 − 5)/1 + 1 = (32 + 4 − 5)/1 + 1 = 31 + 1 = 32
```

Volumen de salida: `32 × 32 × 10` (la profundidad de salida = número de filtros = 10).

4.3 Notación equivalente vista en clase (idéntica, otra forma de escribirla)

```
W_2 = (W_1 - F + 2P)/S + 1
H_2 = (H_1 - F + 2P)/S + 1
D_2 = K
```

Es la misma fórmula reordenada; ambas formas son intercambiables y el profesor las ha usado en distintos exámenes — **debes reconocer ambas.**

---

5. Número de parámetros de una capa convolucional

5.1 Deducción formal

Un filtro de tamaño `F × F` con profundidad de entrada `C` tiene:

```
F · F · C   pesos     +     1 bias     =     (F²C + 1)   parámetros por filtro
```

Si la capa tiene `K` filtros, **cada uno con su propio conjunto independiente de pesos y su propio bias**, el número total de parámetros de la capa es:

```
parámetros_capa = K · (F² · C + 1)
```

Este es el **+1 crítico** que el profesor señala como el error más común del examen: el bias se cuenta **una vez por filtro**, no una vez por capa.

5.2 Notación por capas (subíndice `j`), como aparece en las anotaciones oficiales

Para la primera capa convolucional (entrada con 3 canales RGB):

```
parámetros_1 = K_1 · (3 · L_1² + 1)
```

Para una capa intermedia `j` cualquiera, cuya entrada es el volumen de salida de la capa `j-1` (con profundidad `K_{j-1}`):

```
parámetros_j = K_j · (K_{j-1} · L_j² + 1)
```

Donde `K_{j-1}` es el número de mapas de activación (canales) que llegan de la capa anterior, y `L_j` es el tamaño espacial del filtro de la capa `j`.

Total de parámetros de toda la parte convolucional de la red:

```
parámetros_totales = Σ_j  K_j · (K_{j-1} · L_j² + 1)
```

5.3 ¿Por qué la CNN tiene tan pocos parámetros comparada con una MLP?

Este número **no depende del tamaño de la imagen de entrada**, solo del tamaño del filtro y de la profundidad de los volúmenes contiguos. Esto contrasta radicalmente con una capa densa (MLP), cuyo número de parámetros crece con el **producto** de neuronas de entrada y de salida (ver comparación cuantitativa, sección 12). Esta reducción es consecuencia directa de dos propiedades estructurales de la convolución: **conexiones dispersas** y **pesos compartidos** (sección 8).

5.4 Ejemplo resuelto (mismo ejemplo de la sección 4.2, verificación cruzada)

Datos: filtros `K = 10`, tamaño `F = 5×5`, profundidad de entrada `C = 3`.

```
parámetros_por_filtro = 5·5·3 + 1 = 75 + 1 = 76
parámetros_capa = 10 · 76 = 760
```

---

6. Capas de pooling (submuestreo)

6.1 Definición formal

El **pooling** (o *downsampling*) es una operación de **reducción de la resolución espacial** de un mapa de activación, aplicada de forma **independiente a cada canal** (no mezcla información entre canales), que **no tiene pesos ni sesgos aprendibles**: es una función aritmética fija.

```
Entrada: n_H × n_W × n_C
Salida:  n_H_out × n_W_out × n_C          (¡la profundidad NO cambia!)
```

con las **mismas fórmulas de tamaño de salida** que la convolución (sustituyendo `F` por el tamaño de la ventana de pooling `f_p` y usando el stride del pooling `S_p`, típicamente **sin padding**, `P = 0`):

```
n_H_out = ⌊(n_H_in - f_p)/S_p⌋ + 1
n_W_out = ⌊(n_W_in - f_p)/S_p⌋ + 1
```

6.2 Tipos de pooling

Tipo	Operación	Efecto
Max pooling	Toma el valor **máximo** de cada ventana	Preserva la característica más fuerte/activa; actúa como supresor de ruido
Average pooling	Toma el **promedio** de cada ventana	Suaviza, conserva información de toda la ventana
Global pooling	Una sola ventana = todo el mapa (`f_p = n_H = n_W`)	Colapsa cada canal a un único escalar; el *average global* puede interpretarse como una regresión sobre ese canal

6.3 Ejemplo resuelto (formato de clase, matriz 4×4, max pooling 2×2, stride 2)

```
Entrada (4×4):
2  3  1  9
4  7  3  5
8  2  2  2
1  3  4  5

Max-pooling (ventanas 2×2, stride 2, sin traslape porque S = f_p):

Ventana superior-izq. {2,3,4,7} → máx = 7
Ventana superior-der. {1,9,3,5} → máx = 9
Ventana inferior-izq. {8,2,1,3} → máx = 8
Ventana inferior-der. {2,2,4,5} → máx = 5

Salida (2×2):
7  9
8  5
```

Nota clave (examen): cuando `S_p = f_p` (stride = tamaño de la ventana de pooling), **no hay traslape** entre ventanas consecutivas — es la configuración estándar del pooling en la práctica y la que usa el profesor en sus ejemplos.

6.4 Por qué el pooling es necesario

- Reduce drásticamente el número de valores que llegan a las capas siguientes (y, por tanto, la carga computacional y el número efectivo de conexiones que la MLP final tendría que procesar tras el *flatten*).
- Aporta una forma de **invarianza local a pequeños desplazamientos**: si el patrón detectado se mueve unos pocos píxeles, el máximo de la ventana probablemente sigue siendo el mismo.
- «**Especializa**»: filtra información irrelevante y conserva solo la señal más fuerte de cada región (en el caso de max pooling).

6.5 Lo que el pooling NO hace (trampas de examen)

- **No aprende pesos.** Es una operación fija, determinada únicamente por sus hiperparámetros (tamaño de ventana y stride).
- **No reduce la profundidad (número de canales).** Solo reduce alto y ancho.
- No debe confundirse con una convolución 1×1 (sección 7), que sí puede reducir la profundidad y sí tiene pesos.

---

7. Convoluciones 1×1

Una convolución con filtro de tamaño `1×1` (y profundidad igual a la de la entrada, `n_C`) realiza, en cada posición espacial, una combinación lineal **a través de los canales** en ese píxel:

```
z(i,j) = Σ_c  K(1,1,c) · X(i,j,c) + b
```

Propiedades clave (pregunta de examen frecuente):

- **Sí puede reducir (o aumentar) la profundidad** `n_C` del volumen, eligiendo el número de filtros `K < n_C` (o `K > n_C`).
- **No afecta** `n_H` ni `n_W` cuando se usa con `stride = 1` y sin padding (`O = N - 1 + 1 = N`).
- A diferencia del pooling, **sí tiene parámetros aprendibles**: `K · (1·1·n_C + 1) = K(n_C + 1)`.

Contraste formal con el pooling (tabla resumen para el examen):

Operación	Reduce `n_H, n_W`	Reduce `n_C`	Tiene parámetros
Pooling	Sí	**No**	**No**
Convolución 1×1 (stride 1)	**No**	Sí	**Sí**
Convolución `F×F`, `F>1`	Sí (según `S`, `P`)	Sí (según `K`)	Sí

8. Por qué funciona una CNN: las tres propiedades estructurales

Este bloque fue marcado explícitamente por el profesor como **tema de examen ineludible** («NO OLVIDAR PORQUE LO VA A PREGUNTAR»).

8.1 Conexiones dispersas (*sparse connectivity*)

En una MLP, cada neurona de salida está conectada a **todas** las neuronas de entrada (conexión densa, «producto cruz»: cada salida depende de cada entrada). En una CNN, cada valor del mapa de activación depende **únicamente de la ventana local** (el campo receptivo) cubierta por el filtro — el resto de píxeles de la imagen **no interviene** en ese cálculo.

```
MLP:  cada z_j depende de TODAS las n_entrada entradas
CNN:  cada z(i,j) depende solo de F·F·C entradas (la ventana local)
```

Esto reduce drásticamente tanto el número de conexiones como el número de parámetros.

8.2 Pesos compartidos (*parameter sharing*) → equivarianza traslacional

El **mismo filtro** (mismos pesos `K` y mismo bias `b`) se reutiliza en **todas** las posiciones de la imagen al deslizarse. Es decir: «mismo `W`, distinta entrada» en cada posición.

Consecuencia matemática — equivarianza traslacional: si el patrón de interés (por ejemplo, un borde) se desplaza en la imagen de entrada, el mapa de activación de salida muestra ese mismo patrón detectado, desplazado la misma cantidad. Formalmente, si `T` es un operador de traslación:

```
Convolución( T(X) )  =  T( Convolución(X) )
```

Esto permite que un filtro entrenado para detectar, por ejemplo, un borde vertical en la esquina superior-izquierda de una imagen, **también lo detecte** si aparece en cualquier otra posición — sin necesidad de aprender un filtro distinto para cada ubicación posible (como tendría que hacer una MLP densa).

8.3 Reducción del número de parámetros

Consecuencia directa de 8.1 y 8.2: el número de parámetros de una capa convolucional

depende **solo** de `(F, C, K)` y **no** de `(n_H, n_W)` — es decir, no crece con el tamaño de la imagen, mientras que en una MLP el número de parámetros de la primera capa crece linealmente con el número total de píxeles de entrada (ver comparación cuantitativa, sección 12).

8.4 Respuesta modelo (para pregunta abierta de examen)

> «Una CNN reemplaza las conexiones densas de la MLP por conexiones locales y pesos compartidos mediante la operación de convolución. Cada filtro tiene la misma profundidad que la entrada y se desliza generando un mapa de activación; al reutilizar los mismos pesos en cada posición, la red adquiere equivarianza traslacional (detecta el mismo patrón sin importar dónde aparezca) y reduce drásticamente el número de parámetros respecto de una capa densa equivalente, ya que dicho número depende únicamente del tamaño del filtro, la profundidad de entrada y el número de filtros — no del tamaño espacial de la imagen.»

---

9. Arquitectura completa de una CNN

9.1 Las dos partes de una CNN (distinción de examen)



1. **Parte convolucional:** extrae características (*features*) preservando la información de posición relativa entre ellas; combina bloques `Conv → activación → Pooling` de

forma repetida. Las capas más profundas capturan patrones cada vez más abstractos y semánticos (bordes → texturas → partes → objetos).

2. **Aplanado (*flatten*):** convierte el último volumen 3D en un vector 1D.
3. **Parte de clasificación:** típicamente una MLP con capa de salida `softmax` para clasificación multiclase (aunque, formalmente, cualquier clasificador podría recibir el vector de características aplanado).

9.2 Funciones de activación típicas en CNN

Función	Fórmula	Rango	Uso típico
ReLU	`max(0, z)`	`[0, ∞)`	Capas convolucionales / MLP profundas (estándar en CNN)
Sigmoide	`1/(1+e^{-z})`	`(0, 1)`	Salida binaria/probabilidad
Softmax	`e^{z_i} / Σ_k e^{z_k}`	distribución de probabilidad	Salida multiclase

9.3 Hiperparámetros a fijar por tipo de capa (resumen de clase 12)

Tipo de capa	Hiperparámetros
Convolucional	número de capas, función de activación, número de filtros `K` (convención: múltiplos de 2, p. ej. 32, 64, 128…), tamaño del filtro `F`, stride `S`, padding `P`
Pooling	tamaño de la ventana `f_p` (ventanas grandes, p. ej. 5×5, deterioran el desempeño), stride `S_p` (usualmente `S_p = f_p`)
MLP final	número de capas y neuronas, función de error (divergencia), función de activación de salida

---

10. Forward propagation formal

10.1 Notación general por capas

Siguiendo la notación vectorial usada en el curso (coherente con el forward de la MLP, generalizado a volúmenes):

```
Capa convolucional l:
  Z^[l] = Conv( A^[l-1], W^[l] ) + b^[l]
  A^[l] = g( Z^[l] )                          (g = función de activación, p.
ej. ReLU)

Capa de pooling l:
  A^[l] = Pool( A^[l-1] )                     (sin parámetros W, b)

Capa densa final (tras flatten):
  z_j^(k) = Σ_i  w_ij^(k) · y_i^(k-1)
  y_j^(k) = f_k( z_j^(k) )
```

10.2 Regla mnemotécnica del curso

> «Todo modelo se expresa como `y = f(WX + b)` y se entrena minimizando una función de costo mediante gradiente descendiente y regla de la cadena (backpropagation).» — Esta es la idea rectora repetida en todo el curso: regresión → regresión logística → MLP → CNN → RNN son variaciones de la misma maquinaria (hipótesis parametrizada + costo + gradiente descendiente), cambiando solo **la estructura de las conexiones** y **la forma del dato** (vector, volumen, secuencia).

---

11. Backpropagation en una CNN

11.1 Qué se aprende y qué no

- **Se aprende:** los pesos `W` y sesgos `b` de **cada filtro convolucional**, y los pesos/sesgos de la MLP final.
- **No se aprende:** nada en las capas de pooling (no tienen `W` ni `b`; son funciones fijas).

11.2 Principio general (idéntico al de la MLP)

1. **Forward pass:** se calcula la salida de la red y se compara con la salida deseada mediante una **función de divergencia/costo** (que, formalmente, **no forma parte de la arquitectura** — se mide «afuera» de la red).
2. **Inicialización del gradiente** (**tema declarado nivel examen, incluso vigente en arquitecturas modernas como Transformers**): el reparto del error hacia atrás **comienza** calculando la derivada de la divergencia respecto de la salida de la red, `∂Div/∂y_salida`.
3. **Backward pass:** ese gradiente se propaga hacia atrás capa por capa mediante la **regla de la cadena**, usando:

```
  ∂z/∂y = W        (para pasar de la salida de una capa a la entrada, en
capas densas)
  ∂y/∂z = f'(z)     (derivada de la función de activación)
```

4. Para las capas convolucionales, el gradiente respecto a los pesos del filtro y respecto al sesgo se calcula, para cada posición `(i,j)` donde el filtro fue aplicado, como una convolución/correlación entre la entrada de esa capa y el gradiente que llega de la capa siguiente (análogo matemático de `∂Div/∂w` en la MLP, pero sumado sobre todas las posiciones donde ese peso —compartido— participó).

11.3 La máscara de pooling (backprop del pooling — pregunta segura de examen)

El pooling no tiene pesos que actualizar, pero **sí** debe **enrutar correctamente el gradiente** hacia la capa anterior. Para lograrlo, **durante el forward** se debe **almacenar una máscara** que registre qué posición(es) de la ventana contribuyeron a la salida:

- **En max pooling:** la máscara marca **únicamente la posición del valor máximo**. En el backward, **todo el gradiente entrante regresa a esa única posición**; el resto de posiciones de la ventana reciben gradiente cero.

```
  Ejemplo: ventana 2×2 de max-pooling con salida 7 (proveniente de la
posición (1,2) de la ventana).
  Si el gradiente que llega a esa salida es δ, entonces:

    δ  0          (el gradiente completo va a la posición del máximo;
    0  0           las demás posiciones reciben 0)
```

- **En average pooling: todas** las posiciones de la ventana participaron (con igual peso, `1/(f_p·f_p)`) en el forward, así que en el backward el gradiente entrante `δ` se **reparte equitativamente** entre todas las posiciones de la ventana:

```
  δ/4  δ/4          (ejemplo con ventana 2×2 → f_p² = 4)
  δ/4  δ/4
```

> **Respuesta modelo (formato exacto del profesor, para copiar en el examen):** «En el retropropagado de una capa de pooling no existen pesos que actualizar, pero sí debe enrutarse el gradiente. Para ello, durante el paso hacia adelante se almacena una máscara que indica qué posición contribuyó a la salida: en max pooling, el gradiente regresa únicamente al elemento máximo; en average pooling, se distribuye equitativamente entre todos los elementos de la ventana.»

11.4 Actualización de pesos

```
w_ij^(k)  ←  w_ij^(k)  −  (α/M) · ∂Error/∂w_ij^(k)
```

donde α es la tasa de aprendizaje y M es el tamaño del lote (*batch*) usado para normalizar el gradiente acumulado.

11.5 Medición de perturbaciones entrada-salida (jacobiano — conexión con clase 10)

La sensibilidad de la salida de la red ante pequeñas perturbaciones en la entrada se mide mediante el **jacobiano**: la matriz de todas las derivadas parciales de cada salida respecto de cada entrada. En una CNN, esto se calcula acumulando, capa por capa desde la entrada hasta la salida, las **perturbaciones locales** que cada operación (convolución, activación, pooling) introduce — es decir, aplicando la regla de la cadena de forma sucesiva a través de toda la red, tal como en la pregunta de examen «¿cómo se pueden medir las perturbaciones en la salida de una red neuronal ante pequeñas perturbaciones en la entrada?» (respuesta correcta: midiendo las perturbaciones que cada función genera individualmente en el camino desde la entrada hasta la salida).

---

12. Comparación cuantitativa MLP vs. CNN

12.1 El ejemplo canónico del curso (imagen a color sin usar CNN)

Si una imagen a color de $N \times N \times 3$ se conecta de forma **densa** (MLP) a una primera capa oculta de h neuronas:

```
parámetros_MLP = (N · N · 3) · h + h
               = h · (3N² + 1)
```

Este número crece **cuadráticamente** con el tamaño de la imagen N, y es independiente de cualquier estructura espacial (todas las conexiones se tratan igual, sin importar si dos píxeles están cerca o lejos).

Ejemplo resuelto (300×300, 100 neuronas, formato de examen):

```
entrada = 300 · 300 · 3 = 270 000
parámetros = 270 000 · 100 + 100 = 27 000 000 + 100 = 27 000 100
```

12.2 El mismo volumen usando una capa convolucional

```
parámetros_CNN = K · (F² · C + 1)
```

Este número **no depende de** `N` (el tamaño de la imagen), solo de `F`, `C` y `K`. Por eso una capa convolucional con, por ejemplo, `K=50` filtros de `7×7×3` tiene apenas **7 400** parámetros — miles de veces menos que la MLP densa equivalente — **sin importar si la imagen es de 300×300 o de 3000×3000.**

12.3 Tabla comparativa formal

Aspecto	MLP (capa densa)	CNN (capa convolucional)
Tipo de conexión	Densa (todas contra todas)	Local / dispersa
Operación	Producto cruz (matriz completa)	Producto punto local (convolución)
Pesos	Un peso distinto por cada par (entrada, salida)	Mismo filtro compartido en toda la imagen
Nº de parámetros	Crece con `n_entrada · n_salida` (∝ `N²`)	Depende solo de `(F, C, K)`, independiente de `N`
Estructura espacial	Se pierde (la entrada se trata como vector plano)	Se preserva explícitamente
Equivarianza traslacional	No	Sí

---

13. Banco de ejercicios resueltos (todos los patrones del examen)

A continuación se resuelven, con la misma fórmula y el mismo procedimiento, **todos los tipos de problema numérico de CNN** que han aparecido en el banco de preguntas de exámenes anteriores. Domina estos patrones y podrás resolver cualquier variante numérica del examen.

Patrón A — Parámetros de una capa densa (MLP) conectada a una imagen a color

Fórmula: `parámetros = (n_H · n_W · 3) · h + h`

#	Imagen	Neuronas `h`	Cálculo	Respuesta
Q12	300×300×3	100	`270000·100+100`	**27 000 100**
Q30/Q91-análogo	400×400×3	100	`480000·100+100`	**48 000 100**
Q91	500×500×3	50	`750000·50+50`	**37 500 050**

(Nótese: en este patrón el filtro «cubre toda la imagen», es decir, se trata de una capa completamente conectada, no de una convolución — la trampa habitual es aplicar por error la fórmula de parámetros convolucionales.)

Patrón B — Parámetros de una capa convolucional

Fórmula: `parámetros = K · (F² · C + 1)`

#	Entrada	Filtros	Cálculo	Respuesta
Q116	300×300×3	K=50, F=6×6	`50·(36·3+1)=50·109`	**5 450**
Q138	300×300×3	K=50, F=7×7	`50·(49·3+1)=50·148`	**7 400**
Q140	400×400×3	K=40, F=5×5	`40·(25·3+1)=40·76`	**3 040**

Verificación paso a paso de Q140 (formato de examen):

```
parámetros por filtro = 5·5·3 + 1 = 75 + 1 = 76
parámetros totales    = 40 · 76 = 3 040
```

Patrón C — Tamaño (volumen) de salida de una convolución

Fórmula: `O = ⌊(N + 2P - F)/S⌋ + 1`, volumen = `O × O × K`

#	Entrada	Filtro/Stride/Padding	Cálculo	Respuesta
Q117	63×63×16	K=32, F=7×7, S=2, P=1	`(63+2-7)/2+1 = 58/2+1 = 30`	**30×30×32**
Ejemplo guía	32×32×3	K=10, F=5×5, S=1, P=2	`(32+4-5)/1+1 = 32`	**32×32×10**

Verificación paso a paso de Q117:

```
O = (63 + 2·1 - 7)/2 + 1 = (63+2-7)/2 + 1 = 58/2 + 1 = 29 + 1 = 30
Volumen de salida = 30 × 30 × 32   (32 = número de filtros K)
```

Patrón D — Padding necesario para conservar el tamaño (`S = 1`)

Fórmula: `P = (F - 1)/2`

#	Entrada	Filtro	Cálculo	Respuesta
Q32	60×60×14, K=25	F=3×3, S=1	`(3-1)/2 = 1`	**P = 1**
Q118	65×65×16, K=25	F=5×5, S=1	`(5-1)/2 = 2`	**P = 2**

Verificación paso a paso de Q118: se busca `P` tal que `O = 65`:

```
65 = (65 + 2P - 5)/1 + 1
65 = 61 + 2P
2P = 4  →  P = 2
```

(equivalente y más rápido: `P = (F-1)/2 = (5-1)/2 = 2`)

Patrón E — Dimensión del volumen tras aplicar solo padding (sin convolución)

Fórmula: dimensión resultante = `N + 2P` en alto y ancho; **la profundidad no cambia.**

#	Entrada	Padding	Cálculo	Respuesta
Q92	17×17×8	P=2	`17+2·2=21`	**21×21×8**

Patrón F — Tamaño de salida de una capa de pooling

Fórmula: `O = ⌊(N - f_p)/S_p⌋ + 1`**; la profundidad se mantiene igual a la entrada.**

#	Entrada	Pooling	Cálculo	Respuesta
Q93	64×64×16	f_p=2, S_p=2 (max-pool)	`(64-2)/2+1 = 32`	**32×32×16**

Patrón G — Convolución 1×1 y pooling: qué dimensiones puede reducir cada uno (Q31)

- Una capa convolucional 1×1 (con stride 1, sin padding) **puede reducir** `n_C` (eligiendo `K < n_C`), pero **no** afecta `n_H` ni `n_W`.
- Una capa de pooling **puede reducir** `n_H` **y** `n_W`, pero **nunca** `n_C`.

(Respuestas correctas en el banco: «puede utilizar una capa pooling para reducir `n_H, n_W`, pero no `n_C`» y «puede utilizar una capa convolucional de 1×1 para reducir `n_C`, pero no `n_H, n_W`» — ambas verdaderas simultáneamente.)

Patrón H — Compartición de parámetros (Q94)

Afirmaciones verdaderas sobre *parameter sharing*:

- Permite reconocer un patrón **independientemente de su posición** en la imagen (equivarianza traslacional). ✓
- **Reduce el número total de parámetros**, y con ello, el riesgo de sobreajuste (*overfitting*). ✓

Afirmaciones falsas (trampas):

- «Permite compartir los parámetros de una tarea en otra tarea distinta (*transfer learning*)» → **esto es un concepto diferente** (transferencia de aprendizaje, no compartición de parámetros dentro de la misma capa).
- «Permite al gradiente descendente fijar esos parámetros compartidos en cero» → falso; la compartición no tiene relación con anular pesos.

---

14. Preguntas conceptuales verdadero/falso explicadas

Estas son afirmaciones textuales (o muy cercanas) del banco de exámenes, con su valor de verdad justificado formalmente.

#	Afirmación	V/F	Justificación
Q31b	Se puede usar una capa pooling para reducir `n_H, n_W`, pero no `n_C`	**V**	El pooling opera canal por canal; nunca mezcla ni reduce la profundidad
Q31d	Se puede usar una convolución 1×1 para reducir `n_C`, pero no `n_H, n_W`	**V**	Con stride 1 y sin padding, un filtro 1×1 preserva alto/ancho exactamente
Q66c / Q115a	Las CNN tienen la ventaja de conservar la relación **espacial** de los objetos en la imagen	**V**	Es la propiedad central de la convolución: preserva la topología 2D de la entrada
Q66d / Q115b	Las CNN tienen la ventaja de conservar la relación **temporal** de los objetos en el tiempo	**F**	Esa es una propiedad de las **RNN**, no de las CNN estándar (las CNN no tienen recurrencia)
Q66b	Las CNN necesitan explícitamente que las entradas sean imágenes 3D	**F**	Las CNN operan sobre volúmenes de cualquier profundidad (incluida profundidad 1, escala de grises); el «3D» no es un requisito arquitectónico
Q13	Dado que las capas de pooling no tienen parámetros, ¿estas no afectan el cálculo en backpropagation?	**F**	Aunque no tienen parámetros que actualizar, **sí participan activamente** en backprop: enrutan el gradiente mediante la máscara (sección 11.3)
Q67a	Debido a que el mapa de características siempre es más pequeño que la entrada, para evitar que se reduzca usamos padding	**V**	Es exactamente la motivación del padding «same» (sección 3.2)
Q67c	Cada capa de pooling con stride `S` reduce el tamaño de los mapas en un factor `N×S` (`N`=número de filtros)	**F**	El factor de reducción depende del tamaño de ventana y del stride del **pooling**, no del número de filtros de una capa convolucional distinta
Q67d	Los filtros dentro de una capa de convolución pueden ser de tamaños diferentes y variar de capa a capa	**F** (parcial)	Dentro de **una misma capa**, todos los filtros comparten el mismo tamaño `F×F`; lo que sí puede variar es el tamaño de filtro **entre capas distintas**
Q94a,c	Parameter sharing permite equivarianza traslacional y reduce el número de parámetros (con ello, reduce overfitting)	**V**	Ver sección 8.2 y 8.3

#	Afirmación	V/F	Justificación
Q152a	Para evitar que el mapa se reduzca (porque siempre es más pequeño que la entrada), usamos padding	**V**	Motivación estándar del padding

15. Hoja de fórmulas (cheat-sheet) para el examen

```
CONVOLUCIÓN
z(i,j) = Σ_u Σ_v Σ_c  K(u,v,c)·X(i+u,j+v,c) + b

TAMAÑO DE SALIDA
O = ⌊(N + 2P - F) / S⌋ + 1        Volumen salida = O × O × K

PADDING PARA CONSERVAR TAMAÑO (S=1)
P = (F - 1) / 2

PADDING PARA TAMAÑO OBJETIVO O (S cualquiera)
P = [ (O-1)·S + F - N ] / 2

PARÁMETROS DE UNA CAPA CONVOLUCIONAL j
parámetros_j = K_j · ( K_{j-1} · L_j² + 1 )
(primera capa: K_1 · (3·L_1² + 1), con 3 = canales RGB)

PARÁMETROS DE UNA CAPA DENSA CONECTADA A UNA IMAGEN (SIN CNN)
parámetros = (n_H · n_W · C) · h + h

POOLING (sin parámetros; NO cambia la profundidad)
O_pool = ⌊(N - f_p) / S_p⌋ + 1

ACTUALIZACIÓN DE PESOS
w ← w − (α/M) · ðError/ðw
```

Checklist de variables que SIEMPRE debes identificar antes de calcular:

- [ ] `N` (o `n_H`, `n_W`): tamaño espacial de la entrada
- [ ] `C`: profundidad/canales de la entrada (¡no confundir con `K`!)
- [ ] `F`: tamaño del filtro (asumido cuadrado)
- [ ] `K`: número de filtros de la capa (= profundidad de la salida)
- [ ] `S`: stride
- [ ] `P`: padding

☐ ¿Es una capa **convolucional** o una capa **densa (MLP)** aplicada directamente a la imagen? → determina qué fórmula de parámetros usar

☐ ¿Es una capa de **pooling**? → recordar que no tiene parámetros y no cambia la profundidad

---

16. Errores comunes y trampas del profesor (recopilación explícita)

1. **Olvidar el `2P`** en el numerador de la fórmula de tamaño de salida.
2. **Olvidar el `+1` del bias** al contar parámetros (se cuenta un bias **por filtro**, no uno por capa).
3. **Confundir profundidad del filtro con número de filtros:** el filtro siempre tiene la misma profundidad que la entrada (`C`); el número de filtros `K` determina la profundidad de la **salida**.
4. **Atribuir pesos al pooling.** El pooling nunca aprende parámetros.
5. **Olvidar la máscara de pooling en el backward** (o no distinguir el tratamiento distinto entre max-pooling y average-pooling).
6. **Aplicar conexiones densas (producto cruz) donde corresponde producto punto local** (confundir la lógica de la MLP con la de la CNN).
7. **Pensar que el padding y el número de filtros son intercambiables o irrelevantes** — son hiperparámetros independientes que deben calcularse por separado.
8. **Aplicar la fórmula de parámetros de capa convolucional a una capa densa** (o viceversa) — identifica siempre primero de qué tipo de capa se trata (patrón A vs. patrón B de la sección 13).
9. Asumir que el filtro o la imagen podrían no ser cuadrados: **en este curso, todo se asume cuadrado** (`F×F`, `N×N`), salvo que el enunciado indique explícitamente lo contrario.

---

17. Glosario formal obligatorio (usar esta terminología exacta en el examen)

Término formal (usar este)	Equivalente en inglés	Evitar decir
Mapa de activación	*feature map* / *activation map*	«la imagen que sale»
Campo receptivo	*receptive field*	«ventanita»
Conexiones dispersas	*sparse connectivity*	—
Pesos compartidos	*weight/parameter sharing*	—
Equivarianza traslacional	*translational equivariance*	«que reconoce en cualquier lado»

Término formal (usar este)	Equivalente en inglés	Evitar decir
Submuestreo / agrupamiento	*pooling* / *downsampling*	—
Relleno	*padding*	—
Paso	*stride*	—
Propagación hacia adelante	*forward pass*	—
Retropropagación	*backpropagation*	—
Regla de la cadena	*chain rule*	—
Aproximador universal de funciones	*universal function approximator*	—
Función de divergencia / de costo / de pérdida	*loss* / *cost* / *divergence function*	—

Resumen ejecutivo (una página, para repaso final)

1. **Convolución** = producto punto local entre un filtro `F×F×C` y una ventana de la entrada, más bias, deslizado sobre toda la imagen.
2. **Tamaño de salida:** `O = (N + 2P - F)/S + 1`.
3. **Padding para conservar tamaño (S=1):** `P = (F-1)/2`.
4. **Parámetros de una capa convolucional:** `K·(F²·C + 1)` — el bias se cuenta una vez por filtro.
5. **Pooling:** reduce `n_H, n_W`, **nunca** `n_C`; sin parámetros; requiere máscara para el backward (max → todo el gradiente al máximo; average → se reparte equitativamente).
6. **Conv 1×1:** reduce `n_C`, nunca `n_H, n_W`; sí tiene parámetros.
7. Las CNN son eficientes porque combinan **conexiones dispersas + pesos compartidos**, lo que produce **equivarianza traslacional** y un número de parámetros **independiente del tamaño de la imagen** — a diferencia de una MLP densa, cuyos parámetros crecen con `N²`.
8. Arquitectura: `[Conv → activación → Pooling] × n → Flatten → MLP → Softmax`.
9. Backprop en CNN sigue el mismo principio que en MLP (regla de la cadena, inicialización del gradiente en la salida), con la particularidad de la **máscara de pooling**.